UNIVERSITY OF ZIMBABWE

NAME : BLESSED MAVHEMWA

REG NUMBER : R2424219

PROGRAM : HDSC

FACULTY : SCIENCE

MODULE : DATABASE DESIGN AND DEVELOPMENT

LAB 5 : CONNECTING A DATABASE TO FRONT-END APPLICATION

Objective:

The main objective of this lab was to build a web-based application that interacts with the database which was created beforehand. The application is to connect to the University_Clubs MySQL database and execute SQL queries such as viewing, inserting and modifying data. The lab demonstrates the interaction of a back-end database with a front-end web interface.

Procedure

For this labwork, I used the Python programming language and implemented the Flask web framework with Jinja2 templating for the html codes. The procedures where as follows:

i. **Database Connection and Configuration**

My first step was establishing a connection to the MySQL database. A configuration dictionary stores the connection parameters such as host type, database name which was University_Clubs, user which was 'root' and password which was 'Netone2004#*'. A helper function 'get_connection()' attempts to connect and returns a connection object or 'None' on failure. This function is reused throughout the application to obtain a fresh connection for each database operation. All this is shown in the code snippet below:

```
DB_CONFIG = {
    'host': 'localhost',
    'database': 'University_Clubs',
    'user': 'root',
    'password': 'Netone2004#*'
}
```

```
# ------------------ Database Helper Functions ------------------
def get_connection():  13 usages
    try:
        return mysql.connector.connect(**DB_CONFIG)
    except Error as e:
        print(f"Connection error: {e}")
        return None
```

ii. **Implementing the Back-end to Database Helper Functions**

All SQL operations were encapsulated in their own dedicated helper functions. Each function opens a connection, executes a query, process the result then closes the connection. Error handling was included to catch and print database errors. These functions are listed below and their designated functions:

- get_all_club() – This function retrieves data from the Club table.

- get_club_details(club_id) – This function fetches detailed information about a specific club and joins the instructor and users to obtain the moderator's name.
- add_membership(…) – this function inserts a new mwembership record with the current date.
- get_audit_log() – this function returns all the rows from InstructorAudit table which is created by a trigger
- add_club(…) – This function inserts a new club after validating input. The foreign key constraintsare enforced by the database.

All functions return data in a format easily consumed by Flask templates such as lists of dictionaries. This separation of concerns makes the application easier to maintain and extend.

iii. Application Structure

The entire application is contained within a sngle python file named app.py. HTML templates are embedded directly in the code using a dictionary TEMPLATES and a custom Jinja2 loader. This approach eliminates the need for a separate templates folder and simplifies deployment.

iv. Routes and Views

Flask routes were defined for each desired page:

Routes	Description	Database Function
/	Home page listing all clubs	get_all_clubs()
/students	List all students	get_all_students()
/instructors	List all instructors	get_all_instructors()
/activities	List all activities	get_all_activities()
/add_activity (GET/POST)	Form to add a new activity	add_activity()
/memberships	List all memberships	get_memberships()
/add_membership (GET/POST)	Form to add a new membership	add_membership()
/audit	Display the salary audit log	get_audit_log()
/moderator_budget	Show total budget per moderator	get_moderator_budget_summary()
/club_details_view	Display the ClubDetails database view	get_club_details_view()
/add_club (GET/POST)	Form to add a new club	add_club()

All POST routes validate user input, call the appropriate helper functions and then flash a success or error message.

v. **Template Design**
Templates are written in HTML with Jinja2 syntax. A base template called base.html provides the overall structure of the application that is a left-sidebar navigation menu and a main content area. All other templates extend base.html and override the content block. The sidebar contains links to all major pages thus making navigation intuitive.

Challenges and Solutions

i. **Foreign Key Constraints**
When inserting a new record, the database enforces referential integrity. This caused initial errors when users entered non-existent IDs. The solution was to rely on the database error handling and display a generic "Error adding record" message.

ii. **Auto-increment Handling**
The Activity table doesn't have an AUTO_INCREMENT primary key so the application manually computes the next activity_id by selecting MAX(activity_id) +1. This works for a single user environment but would need adjustment for concurrent use.

iii. **Embedding Templates**
Placing all HTML inside a Python dictionary made the file long but ensured portability. The custom Jinja2 loader was essential to make "render_template()" work without a physical template folder.

iv. **Password Security**
The database password is hard-coded in the script. For a production system, it should be read from environment variables or a configuration file.

Conclusion

This lab successfully demonstrated how to connect a MySQL database to a web-based front end using python and flask. The resulting application provides full CRUD functionality for University_Clubs database, allowing users to view, insert and modify data through an intuitive interface. All major database objects like tables, views triggers and the audit log are exposed and usable.

The experience reinforced key concepts of database connectivity, server side logic and user interface design. The modular helper functions and clean routing make the application easy to extend. The single file deployment is ideal for educational settings and quick demonstrations.

User Manual

i. Requirements

Python 3.8 or higher installed on your system
MySQL Server running with the University_Clubs database already created and populated using the scripts from previous labs.
Required Python packages are flask and mysql-connector-python

ii. Running the Application

- Save the provided app.py file to a folder on your computer.
- Open a terminal in that folder.
- Update the database password in the DB_CONFIG dictionary if necessary
- Run the command: 'python app.py'
- The terminal will show a message like 'Running on "http://127.0.0.1:500/". Open that URL in your web browser.

iii. Using the Application

- **Navigation:** The left sidebar contains links to all main sections which are Home, Students, Instructors, Activities, Membership, Audit Log, Moderator Budget and Club Details View.
- **Home Page:** Lists all clubs with their basic information. From here you can click on a club name to view details.
- **Club Details Page:** Shows the club's budget, meeting schedule and the moderator's name. A link returns to the home page.
- **Students/Instructor Pages:** Displays all individuals with their attributes such as registration number, program and salary.
- **Activities Page:** Lists all activities with the club name, type, date, time and location. A button at the top leads to the "Add Activity" form.
- **Add Activity Form:** Select a club from the dropdown, fill in the necessary activity details and submit by clicking the "Add Activity" button.
- **Membership Page:** shows which students belong to which clubs along with the join date and role. The "Add New Membership" button opens a simple form.
- **Audit Log:** displays records of instructor salary changes which are automatically captured by a database trigger.
- **Moderator Budget:** shows each instructor and the total budget of all clubs they moderate.
- **Club Details View:** presents the pre-joined data from the ClubDetails database view including member count per club. The "Add New Club" button allows adding a club